

Constitutional ARI-RQGM: Governed Co-Evolution for Autonomous Research

Takanori Kotama

v0.9.0, Constitutional RQGM edition, 2026-07-28

Abstract

Autonomous research systems increasingly delegate proposal, evaluation, and writing decisions to model-defined roles. If those roles and their scoring policy can evolve, the system may improve its institution as well as its artefacts; it may also learn to reward itself, conceal failures, or expand its own authority. We present **Constitutional ARI-RQGM**, a governed co-evolution architecture that separates an immutable deterministic kernel from evolvable prompts, components, and utility policies. Institutional changes commit only at epoch boundaries; T16 emergency quarantine forces an immediate boundary with a new fingerprint. At a boundary, adversarial evidence, role-separated prosecution and defence, replay and anchor evaluation, and a fixed T1–T21 transition table determine which candidates may enter service. Registry writes are single-authority, transactions are replayable, and retirement triggers provenance-aware frontier repair without deleting history. The same constitution governs a paper archive in which writer and reviewer roles co-evolve while a fixed, structured numeric claim–evidence gate remains outside their authority. The complete transition table, capability matrix, event contract, recovery procedure, and determinism boundary are specified. All 1,124 directly relevant tests pass, and five fixed fault fixtures are rejected while one clean fixture passes. These results do not establish improved research quality, general detection rates, attribution accuracy, or cost benefit.

1 Introduction

1.1 Motivation

Autonomous research systems promise to shorten the path from a question to an experimentally grounded answer. Their central difficulty is not merely generating code or prose. A useful system must decide what to try, judge noisy outcomes, preserve evidence, recover from failure, and keep a manuscript from claiming more than the run established. These decisions form an institution: proposers, evaluators, adversaries, writers, and judges whose behaviour is largely encoded in prompts and scoring rules.

A fixed institution is reproducible but inherits every weakness of its initial design. An evolvable institution can adapt, but creates a reflexive control problem. A component that benefits from a changed score may supply the evidence for changing it; a reviewer may grade its own successor; a prompt that gains tool access may hide the history that exposes the expansion. Self-improvement is therefore an authority problem as well as an optimisation problem.

1.2 Verified claims as a release condition

Constitutional Autonomous Research Infrastructure (ARI)-RQGM starts from a narrow rule: model outputs may propose institutional change, but no model output may enact it directly. A deterministic kernel validates record shapes, hashes, capabilities, role

separation, transition legality, audit continuity, clean-room inputs, and context scope. A single transition engine owns registry writes. Changes occur only at boundaries; T16 emergency quarantine force-closes the current epoch and opens a newly fingerprinted epoch in the same transaction. A boundary transaction either commits in full or disappears on replay.

The fixed layer also retains the final structured numeric claim–evidence decision. A governed writer and reviewer may compare manuscript alternatives, but neither can modify the verifier that recomputes forward-declared formulas from executed evidence. It is not a general natural-language truth checker. Governance selects what is presented to the gate; it cannot relax the gate.

1.3 Contributions

This report makes five contributions.

1. It defines a three-layer authority model: an immutable constitutional layer, an evolvable institutional layer, and a meta layer that may propose replacements but cannot appoint them.
2. It specifies the complete T1–T21 component-lifecycle table, including the forced emergency boundary for T16, together with the capability matrix, event contract, and deterministic recovery procedure.
3. It closes the accountability loop from a raw artefact attack to defence, adjudication, a validated

attack, and target binding for the epoch-stamped research generator and the currently connected paper roles. Ambiguous provenance remains deliberately targetless.

4. It gives retirement operational meaning through selective erasure, clean-room regeneration, replay evaluation, and provenance-aware repair, including successor-policy re-scoring from stored raw axes.
5. It applies the constitution to a governed paper archive and reports lifecycle and fault-injection evidence from the implementation.

1.4 Scope of this report

The subject is institutional co-evolution, not a catalogue of every execution feature in ARI. We treat the experiment executor, domain tools, and compilation pipeline as an environment that produces and consumes typed artefacts. Their internal scheduling strategies are out of scope. The constitutional claims should hold for any research executor that obeys the record, provenance, and checkpoint contracts.

We distinguish construction guarantees from empirical claims. Single-writer registry authority, legal transition topology, and fail-closed state changes are structural. A local hash chain detects interior modification but not rollback of the whole checkpoint and local pin to an old valid prefix. Improved novelty, research quality, calibration, and cost effectiveness require controlled evidence. Current results are 1,124 passing directly relevant tests, five fixed fault fixtures, and one clean fixture. They do not establish general detection rates, attribution accuracy, research-quality improvement, or cost effectiveness.

1.5 Organisation

Section 2 defines the system and threat model. Section 3 specifies the constitutional lifecycle, co-evolution, paper governance, and observability. Section 4 separates demonstrated behaviour from deferred claims. Section 5 positions the architecture against self-modifying agents, autonomous research systems, and verification work. Section 6 discusses the fixed-constitution trade-off and open scaling questions. The appendices give the complete core specification and reproduce the RQGM-specific runtime prompt seeds verbatim.

2 System and Threat Model

2.1 System boundary and actors

Constitutional ARI-RQGM governs the institution surrounding an autonomous research executor. The executor produces typed research artefacts, measurements, review records, and manuscript candidates inside a checkpoint. The governance system observes

those records and controls which prompt, component, and utility-policy versions may serve. It does not control the host operating system, model weights, or the human who can change the program.

The actors divide into four groups. *Research actors* propose, evaluate, defend, and write. *Judiciary actors* collect evidence, prosecute, defend governance motions, and adjudicate them. *Meta actors* generate candidate prompts and policies or select replay cases. *Fixed actors* validate constitutional rules, apply registry transitions, repair affected records, and verify final claims. An actor may produce a hashed record only within its declared role. The current artifact does not implement digital signatures or role-key management. Producing a recommendation never implies authority to enact it.

2.2 Threat model

The primary adversary is reflexive institutional optimisation: an evolvable role may seek a change that makes its own outputs look better rather than making the research more reliable. Concrete threats include score inflation, metric gaming, evidence omission, prompt injection, collusion between proposer and evaluator, a meta agent requesting broader tools than its incumbent, a judge ruling on its own prosecution, and a retired prompt remaining influential through derived scores.

The model also includes accidental failures. A process may stop between registry events; a model may emit malformed output; an anchor corpus may be absent; a prompt candidate may fail its output schema; or stored evidence may lack the raw axes needed for re-scoring. These failures must not produce a half-applied institution.

The threat model excludes a malicious host administrator, arbitrary modification of the fixed program, compromise of the underlying hash function, rollback of both a checkpoint and its local pin to the same old prefix, secret modification of model weights by the provider, and irreversible external API, device, or production-data operations. Changing the constitutional program is treated as a human amendment, not an attack the running system can prevent.

2.3 Authority and verification boundary

Authority flows in one direction. Model-bearing roles may create proposals, attacks, defences, judgments, and candidates. The deterministic kernel converts none of these directly into registry state. Instead it checks the record, the acting role, the requested edge, epoch timing, capability subset, provenance, and audit continuity. A transition engine is the sole registry writer, and its output is checked again before commit.

Two asymmetric failure rules preserve useful work without allowing an uncertain institutional change. A governance hook on the research path is *fail-open*: its

failure is recorded and candidate generation or local computation may continue only in a side-effect-free sandbox. A state-changing governance operation is *fail-closed*: if its evidence or legality cannot be established, the incumbent institution remains in service. External irreversible actions require a separate fixed gate that fails closed; the present kernel does not enforce that environmental condition.

The final manuscript verifier is part of the fixed boundary. The manuscript forward-declares numeric claim identifiers, formulas, operand locations, units, and tolerances. The verifier recomputes this closed formula vocabulary from execution evidence. It does not infer arbitrary claims from natural language or establish the truth of non-numeric claims or execution records. Writer and reviewer roles cannot change the verifier, its inputs, or its verdict.

2.4 State, provenance, and replay

The checkpoint is the unit of institutional identity. Its append-only events are authoritative; registry, epoch, and erasure snapshots are derived views that can be reconstructed by replay. Every governed record carries the epoch, serving component, prompt hash, and relevant policy hash. Audit events form a predecessor-linked hash chain. Consequently a reader can distinguish what happened, which institution produced it, and which later transition changed its eligibility. This local chain detects interior deletion or modification, and a surviving local boundary pin detects a shorter log. It cannot detect rollback of both log and pin to the same old valid prefix. There is no external signed head, witness checkpoint, or monotonic counter, and the hash is not an author signature.

Epoch transactions use prepare, close, open, and commit events. Replay ignores an incomplete suffix and deterministically re-executes the boundary. Founding registration is similarly write-once: a committed founding transaction is replayed, never repeated. Logical erasure adds staleness and eligibility fields rather than deleting bytes, preserving the evidence needed to audit the retirement itself.

Epoch identity is split before it is composed. A policy fingerprint binds the serving institution and all resolved governance settings, including thresholds and budgets. An execution fingerprint binds the declared model, decoding, tools, environment, and data snapshot. Missing provider or environment revisions are explicitly marked unresolved. The former supports institutional comparisons; the latter exposes when an apparently unchanged policy ran on a different declared execution substrate. Neither turns an opaque model service into a reproducible binary.

2.5 Design principles

Five principles organise the architecture.

1. **Fixed law, evolvable institution.** The constitution, transition topology, and final verification

path are outside the set of runtime-mutable artefacts.

2. **Frozen service epochs.** A serving version changes only at a boundary. T16 emergency quarantine forms an immediate boundary and opens a new epoch fingerprint, so its before/after nodes have distinct institutional identities.
3. **Separation of recommendation and authority.** Proposers, judges, and meta agents emit records. Only the transition engine writes the registry, under kernel validation.
4. **History without inherited influence.** Retirement never deletes evidence, but affected descendants become stale or are re-scored before they may influence future selection.
5. **Claims proportional to evidence.** Structural guarantees are stated as guarantees; mechanism demonstrations are not presented as evidence of improved research quality.

3 Constitutional Governance and Co-Evolution (ARI-RQGM)

ARI's *institution* is the set of prompt-defined roles that propose research directions, review artefacts, judge quality, and write the manuscript. *ARI-RQGM* lets that institution evolve under a constitution that does not. An epoch-based governance layer makes prompts and components compete, attack, defend, and become adopted or retired, while every institutional state change is validated by a deterministic, non-model kernel against rule tables frozen in code. Governance is selected before a run begins, and both the resolved mode and its provenance are recorded in the checkpoint. All changes commit at a boundary; T16 emergency quarantine forces an immediate close/open boundary with a fresh institutional fingerprint.

3.1 Motivation and lineage

The institutional roles are themselves prompt-defined artefacts, so letting them evolve promises the same kind of improvement the search loop extracts from experiments. Unconstrained self-modification, however, is precisely what the Gödel-machine literature warns about: a component that can rewrite its own evaluator will eventually reward itself. The Red Queen Gödel Machine [1] adds the co-evolutionary counterweight — agents improve *because* adversaries and evaluators co-evolve against them — and ARI-RQGM takes its name and its central dynamic from that line of work. But co-evolution alone does not bound the failure modes that matter for a research system: score inflation, metric gaming, evidence fabrication, and collusion between roles.

ARI-RQGM therefore places the co-evolutionary loop under a fixed constitutional layer that never evolves. Its availability policy permits candidate generation and local computation to continue after governance failure only under a side-effect-free sandbox assumption. The kernel does not enforce isolation of external APIs, devices, or production data; irreversible operations need a separate fixed gate. State changes remain fail-closed.

The determinism principle of section 2.5 is narrower than model reproducibility. Given the same saved advisory report, candidate evaluations, registry, and configuration, transition resolution and frontier repair are pure functions. The same committed event sequence reconstructs the same projection. We do not claim that another model call regenerates the same defence, adjudication proposal, or review; the three boundaries are specified in section A.5.

3.2 Three layers, four facades

Governance stratifies the system into three layers with strictly decreasing authority to change things and strictly increasing freedom to be changed.

Layer 0, constitutional (fixed). The deterministic kernel, the fixed result-verification path, and the append-only audit log. These never evolve; the rule tables they enforce live in code, pinned by a constitution hash (section 3.9), and are registered in the component registry for provenance only.

Layer 1, institutional. The evolvable service roles: reviewer, adversary, defender, artefact judge, router, utility policy, and the paper writer/reviewer when the archive phase is active. Their prompts or policy may be replaced through the lifecycle of section 3.8, at epoch boundaries only. The auditor, evidence clerk, and governance judge are also registered and sanctionable, but v1 deliberately gives these judiciary actors no successor-generation path.

Layer 2, meta. The agents that evolve Layer 1 — the prompt and policy mutators, clean-room generator, replay selector, and failure-summary compressor — themselves governed, with strictly less authority than the layer they modify: a meta agent can propose a replacement, never appoint one.

Four facades own the machinery. The *constitutional kernel* validates records, hashes, capabilities, transitions, role separation, erasure, audit-log integrity, clean-room inputs, and context scope — twelve closed entry points, none of which calls a model or reads a clock. The *governance orchestrator* runs the epoch-boundary audit (section 3.5) and produces an advisory report; it never writes a registry. The *registry transition engine* is the sole writer of component and prompt statuses, resolving changes against a fixed transition

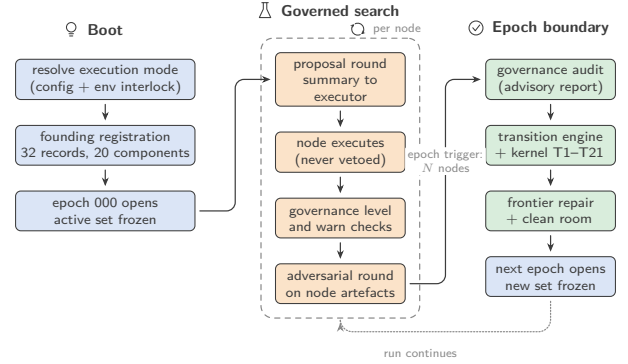


Figure 1: Lifecycle of a governed run. Boot resolves the execution mode, executes the one-time founding registration transaction (32 prompt-or-policy records, 20 components), and freezes the founding active set as epoch 000. During the epoch, per-node hooks record proposals, assign a governance level (L0 fixed to L3 adjudicated), and run an event-driven adversarial round; node execution itself is never vetoed. When the epoch’s node-count trigger fires, the boundary runs the advisory audit, the kernel-validated transition, and frontier repair with clean-room regeneration, then freezes the next epoch’s active set and the search continues.

table. The *frontier repair engine* restores the search frontier after a retirement (section 3.5). Around them sit the per-node services: a proposal router, an adversarial round, a prompt-evolution pipeline, a clean-room coordinator, and a budget manager that can degrade any governance decision point without ever touching node execution.

3.3 Boot: mode resolution and founding registration

A governed run boots in three steps (fig. 1). First, *mode resolution*: the mode is a two-key interlock — a mode selector and a redundant enable flag must agree, with environment overrides validated before assignment — and any mismatch warns and leaves the constitutional layer inactive rather than guessing. The resolved mode and its provenance (configuration, environment, or resume) are recorded in the checkpoint.

Second, the runtime attaches governance as best-effort hooks around the research executor. The executor retains authority over scientific work; the hooks observe typed proposals, evaluations, and artefacts without acquiring authority to veto their execution.

Third — on a fresh governed checkpoint only — the *founding registration* transaction populates the institution. From two tables frozen in code, one prepare event is followed by 32 prompt registrations and 20 component registrations, then a single commit. The 20 founding components are the research generator, seven exploration-adversary variants, artefact judge, auditor, clean-room generator, defender, evidence clerk, failure-summary compressor, governance judge, policy mutator, prompt mutator, proposal router, replay selector, and utility policy. The

32 founding prompts comprise 31 committed templates plus the configuration- derived utility-policy prompt. The paper-archive phase registers three further prompts and components — paper writer, paper reviewer, and the paper-self-preference adversary — for totals of 35 and 23. Each prompt is registered with the hash of its template bytes. The transaction then freezes the founding active set — active components, active prompt hashes, and the utility policy — into the immutable state of epoch 000. Its policy fingerprint also binds the resolved governance settings, while its separately reported execution fingerprint binds declared model, decoding, tool, environment, and data identities; missing external revisions remain marked unresolved. The composite deliberately excludes timestamps. Founding is write-once: on resume the registries are reconstructed by replaying the event log, a committed registration is never re-run, and a founding transaction interrupted before its commit is invisible and simply re-executed.

3.4 Per-node governance

During an epoch the research executor continues to operate while governance observes it through three best-effort hooks.

Proposals. When the expansion step generates candidate directions, the proposal router archives a full *proposal record* — the complete generation context, on the principle of storing everything — while the search itself receives only a capped *proposal summary view*. The summary is the *only* surface the search sees, so proposal generators can grow richer without widening the interface to the search loop, and the archived records remain available to audits and to replay.

Governance levels. Each evaluated node is assigned a governance level by a pure trigger ladder — from L0 (fixed checks only) to L3 (fully adjudicated) — based on deterministic signals: membership in the frontier’s top scorers, an anomalous score jump relative to the parent, a paper-candidacy flag, a novelty claim, or low evaluator confidence. The level decides how much governance attention the node receives, never whether it runs. Every assignment is appended to the audit log, and the budget manager may cap the effective level when a per-epoch governance budget runs low — degrading governance, not the node. Continuation after governance failure is safe only for side-effect-free candidate generation and local computation; this is an operating condition, not a kernel invariant.

Adversarial rounds. Nodes that trip the same trigger family also receive an *adversarial round*: one of seven exploration adversary types — overclaiming, evidence gaps, metric gaming, reproducibility, prior-art, cost explosion, and prompt injection — attacks the node’s *artefacts*; the paper phase adds an eighth,

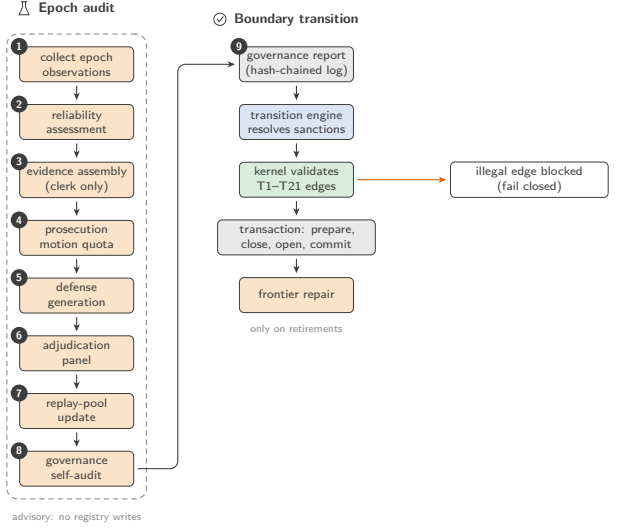


Figure 2: The epoch-boundary pipeline. The nine-step audit is advisory and read-only with respect to registries; its report feeds the registry transition engine, whose resolved transition is validated edge-by-edge by the constitutional kernel against the fixed T1–T21 table before the four-event boundary transaction commits. An illegal edge is blocked fail-closed — the incumbent active set keeps serving and the run continues. Frontier repair runs only when the committed transition retired a component or prompt.

paper-self-preference adversary. Adversaries never attack components; the raw-attack record has no component target by construction. A raw attack is score-inert: only after the defender responds and the artifact judge upholds the attack does a *validated attack record* exist, and only validated attacks feed a bounded utility penalty under the epoch-frozen penalty policy, with the pre-penalty score preserved in additive metric keys. The judge-authored validated record may then bind an implicated role to an epoch-frozen incumbent component for accountability (section 3.7). A judge failure defaults to *invalid* — no penalty. This avoids an unverified sanction but is vulnerable to denial of the adjudication path; fixed adjudication and independent retry are not implemented. Validated attacks also enter the *adversarial replay pool*, the corpus that later tests candidate prompts (section 3.8) — the Red-Queen coupling: today’s validated attacks are tomorrow’s entrance exam. The round is idempotent per node — a marker in the case log ensures a resumed run never re-attacks a node it already processed.

3.5 The epoch boundary

An epoch freezes the institution; the boundary is the only window in which the institution may change. The boundary trigger is node count (ten new nodes per epoch by default), checked on the main thread at the head of each outer search iteration — never with a node in flight — and once more at end of run, so an epoch whose final node arrives in the last iteration still closes transactionally; a trailing epoch below the trigger simply stays open. The end-of-run check exists

because live runs exposed the gap: a boundary reached exactly at the last node would otherwise never fire, leaving the epoch’s audit unwritten.

The audit. The closing epoch is audited first (fig. 2), by a nine-step pipeline with a total deterministic fallback at every LLM-consuming step, so an audit without a model still completes. Observations are collected from the epoch’s records; a reliability assessment scores each governed component; the *evidence clerk* — and only the clerk — assembles evidence bundles, excluding same-role accusations, which are inadmissible by constitution; prosecution follows, where only the *auditor* may file an impeachment motion against a per-epoch motion quota; internal **posted/refunded/forfeited** counters account for that finite allowance but transfer no value and are a rate limit rather than an economic bond; a defense is generated for every motion; an adjudication panel rules; the replay pool is updated; and a *governance self-audit* re-validates the governance actors’ own records through the kernel. These judiciary actors are founding components and can themselves be sanctioned; when the governance judge is the target of a motion it recuses and the deterministic board fallback decides the case. The result is a *governance report* — reliability, motions, defenses, adjudications, candidate evaluations, recommendations, motion-quota accounting — appended to the hash-chained audit log. The report is advisory input to the next stage; the orchestrator that produced it cannot write a registry.

The transition. The registry transition engine resolves the report’s recommendations and the candidate evaluations into one epoch transition — adoptions, sanctions, retirements — as a pure function of the frozen epoch state, the report, the registries, and the configured thresholds: byte-identical on re-run. Components and prompts move through ten statuses (candidate, validated, shadow, probationary active, active, warning, probation, quarantine, retired, banned) along a fixed table of twenty-one rules, T1–T21. The kernel validates every edge of the resolved transition: an edge outside the table is the named violation CK-REG-001; a boundary-only edge stamped mid-epoch without the emergency shape is CK-REG-002; a transition not authored by the transition engine — a judge writing the registry, say — is CK-ROL-901. A blocked transition aborts fail-closed with the incumbent active set still serving; governance is suspended, the run never is. A valid transition commits through a four-event boundary transaction — prepare, close the old epoch, open the new one, commit — so a crash between any two events leaves a prefix that resume discards and re-runs deterministically, and a duplicate commit of the same transition is a guarded no-op. *Emergency quarantine* (rule T16) handles a kernel-critical violation without weakening epoch identity. It force-closes the current epoch and commits the quarantine, close, and opening of a freshly fingerprinted

epoch in one transaction. The prior utility policy is preserved and no replacement is adopted in that emergency transaction. Nodes on the two sides therefore carry different epoch identities; section A.1 gives the complete component-lifecycle table.

Frontier repair. If the committed transition retired a component or prompt, the run’s past contains records produced under an institution now judged defective, and the frontier must not keep paying interest on them. The repair engine starts from records stamped with the retired prompt hash and follows reverse provenance references breadth-first. Only proposal-to-proposal inspiration is declared non-material; unknown edges are material. The default depth cap is eight, with any still-reachable records conservatively invalidated. It marks the affected nodes stale via additive flags (stale, valid-for-frontier, and the erasure reason) persisted through the tree file. Erasure is *logical only*: nothing is physically deleted, so the history remains auditable while every consumer that promotes a node — frontier scoring and expansion, best-node selection, and the paper’s evidence grounding — reads the flags and ignores it; when every candidate is erased, selection reports no winner rather than falling back to contaminated evidence. Repair is role-aware and has two explicit recomputation paths. Utilities that merely *consumed* a retired reviewer’s scored evidence are recomputed from surviving inputs under the *original* epoch’s frozen weights; records the retired role itself produced are invalidated outright. By contrast, when the *utility policy* retires, every stamped node is re-scored under the successor policy by recomposing its stored per-axis raw scores, and the repair event lists those nodes in **policy_rescored_node_ids**. Missing raw axes fail closed to invalidation. This is an operational definition, not an evaluated causal guarantee; semantic precision and over-removal are unknown. If repair itself fails validation, the engine degrades conservatively, ultimately to a drain-only mode in which the run finishes pending work but expands no further — never a crash.

3.6 The evolving score

Every subsection so far has let the institution evolve while treating one thing as fixed: the *score* itself — the utility policy that ranks the frontier, built from an axis basis and its weights, a composite rule, a frontier-scoring rule, a depth penalty, and an exploration constant. Freezing the score for the life of a run is the conservative choice, but it is not the one the Red Queen line of work [1] actually makes: its defining claim is that the ordering objective itself is rewritten at each epoch boundary. Governed utility evolution closes that gap. The utility policy becomes an evolvable role in its own right, with one difference from the prompt roles of section 3.8 — its incumbent is a policy document, not a prompt template. It is frozen within an epoch, so the frontier is ranked by

one fixed rule while an epoch runs, and it is rewritable only at the boundary, by the same transition engine that adopts prompts.

A dedicated proposer mints candidate policies and nothing else. Its four default mutations are pure arithmetic over the boundary’s already-abstract evidence — no model, no clock, no randomness — so the default rewrite is fully reproducible, and it is forbidden from reading the frontier’s current scores: a score tuned to flatter the nodes it has already produced is exactly the self-referential collusion the co-evolutionary design exists to prevent. The proposer holds no registry authority, and it is itself a registered, sanctionable component — there is no absolute ruler over the score any more than over the roles it grades.

Adoption rides one new edge in the transition table. A successor policy that has cleared validation and shadow service and scores at least as well as the incumbent on the frozen replay board *supersedes* it: rule T20, the sole active-to-retired edge, and the one retirement that does not stage through quarantine. The kernel permits it for the utility-policy role alone — every behavioural component keeps the conservative, sanction-only replacement model, and for them a direct retirement is still rejected. Supersession retires the healthy incumbent under its *old* policy hash, and because every node now carries the hash of the policy that scored it, frontier repair (section 3.5) selects every node ranked under the retired policy, not merely the subset that happened to be penalised, and re-composes its stored raw axis scores under the successor. Only a node without recoverable axes is invalidated fail-closed. Adding the edge re-pinned the constitution hash, as an amendment must (section 3.9).

One limit is worth stating as plainly as the mechanism. When the score’s axes are derived dynamically and no fixed per-axis basis is pinned, the “scores at least as well” gate has no stable ordering to compare against, so supersession reduces to legality and non-degeneracy rather than a demonstrated improvement. A gate that can certify a successor as *strictly* better needs an anchored basis — ground truth to score against — which the paper pipeline of section 3.10 supplies but the exploration phase, lacking any external oracle over research quality, does not. This is the honest Red-Queen posture: with no ground truth, the criterion rolls within the space of legal policies rather than climbing a fixed hill, and we do not present it as more than that.

3.7 Binding an attack to an accountable component

The adversarial round of section 3.4 produces, after adjudication, a validated attack record. It first names implicated *roles*; sanction additionally requires resolution to the component that produced the attacked artefact under that epoch.

Each judge-authored record carries, whenever one is resolvable, a single registry-resolvable target: the incumbent that made the decision the attack invali-

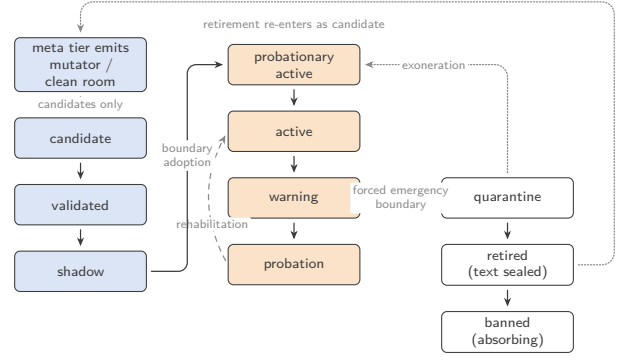


Figure 3: The governed prompt lifecycle. Meta-tier agents (the prompt mutator; after a retirement, the clean-room generator) emit candidates only. A candidate climbs the six-stage validation ladder (static, constitutional, schema dry-run, replay against the adversarial pool, anchor, shadow) and can be adopted only by the transition engine at an epoch boundary, entering service under probation. Sanctions descend through warning and probation; the accented edge is T16 emergency quarantine, which forces an immediate close/open boundary. A retired prompt’s text is sealed; its role is served by the baseline fallback until a clean-room replacement re-enters the lifecycle as a fresh candidate. Rejection edges from candidate and shadow to retired (T2, T5) are omitted for clarity.

dates, resolved when the record is minted against the epoch-*frozen* active set — never a live registry read, which after a boundary adoption would impeach a successor for its predecessor’s defect. Role separation is preserved exactly: the adversary attacks artefacts and only artefacts, and it is the judge’s verdict, not the accusation, that makes an observation accountable, so no actor both accuses and binds; a record that would bind its own adjudicator drops the binding and says nothing rather than raise. When no target resolves, the field is absent and no sanction follows. If multiple roles are implicated, the system emits one record per role referencing the same adjudication; the one-target constraint is an audit format, not a causal guarantee.

The research generator is now a founding component. Before a generated node is recorded or attacked, the runtime stamps its producer component, prompt hash, and epoch exactly once from the frozen active set. Each of the seven exploration attacks binds that generator only when all available provenance agrees; legacy, missing, or mismatched provenance remains targetless, preventing a successor from inheriting a predecessor’s defect. For the paper self-preference attack of section 3.10, one round can bind the manuscript reviewer, the paper writer, or both registered components. The fixed mapping’s attribution accuracy and its treatment of tool failures, data contamination, shared prompts, and genuinely joint causes have not been evaluated.

3.8 Prompt evolution, clean room, and the meta tier

Institutional prompts evolve through the lifecycle of fig. 3. The *prompt mutator* emits candidate templates and nothing else — it holds no registry authority, and there is deliberately no edge from candidate to active, so no output of a model can go straight into service. A candidate must clear six stages: *static validation* (deterministic well-formedness and no in-place mutation of the incumbent), *constitutional validation* (kernel checks on the candidate record), a *schema dry-run* (one injected call must produce a reply parseable under the role’s declared output schema), *replay evaluation* against the adversarial replay pool — candidates are graded on the very attacks that defeated their predecessors — an *anchor evaluation* on a fixed task set where ties favour the incumbent, and finally *shadow* service, in which the candidate runs side by side with the incumbent on sampled traffic with observation-only effect. Only then can the transition engine adopt it at a boundary, and adoption enters service as probationary, not active.

Retirement triggers the strictest path in the system. A retired prompt’s text is sealed: registry views stub it out, and a dedicated access guard denies it at the capability layer, everywhere, forever. The replacement is produced in a *clean room*: the fixed tier assembles a closed input bundle — committed catalogues and abstract failure summaries, never registry prompt text — which the kernel screens before generation (violations are the CK-CLN family, including a word-shingle similarity screen against the retired text at admission), and generation is one-shot, with no tool-bearing loop that could go looking for the sealed text. The regenerated candidate re-enters the lifecycle at the bottom rung; meanwhile the retired role’s slot is served by the built-in baseline, so no governed role is ever vacant. The screen is lexical: semantic paraphrase leakage and false rejection of legitimate role vocabulary have not been measured. Clean-room replacement is therefore a contamination reduction mechanism, not a guarantee of semantic non-inheritance.

The meta tier is governed more tightly than the layer it modifies. Meta agents run in a read-only sandbox behind an allowlisted tool proxy whose only write surface is a synthesized submission tool; their capability flags are deny-by-default with the dangerous flags constant-false; the kernel checks *authority non-expansion* as subset arithmetic — a candidate cannot declare more authority than the serving incumbent it would replace (or a fixed role matrix when no incumbent exists); and every meta output enters the lifecycle at candidate status. The meta tier can propose; it cannot activate. Nor is it failure-gated or silent: the mutator mints candidates at every epoch boundary — calm epochs included — under per-role and per-epoch budget caps, and each boundary’s meta step records an audit-visible summary of its outcome even when it proposes nothing.

Family	What it guards
CK-SCH	Record schema: malformed governance records block; malformed node-side records warn.
CK-HSH	Hash integrity: records, registered prompt hashes, artefact digests.
CK-ACC	Capability: the deny-by-default authority matrix per role and tier.
CK-ROL	Role separation; CK-ROL-901 names the canonical violation — anyone but the transition engine writing the registry.
CK-REG	Transition legality against T1–T21: 001 an edge outside the table, 002 a boundary-only edge attempted mid-epoch.
CK-EPO	Epoch invariance: the active set may not drift inside one fingerprinted epoch.
CK-ERA	Selective erasure: stale records must not reach frontier scoring or best-node selection.
CK-AUD	Audit-log integrity: the event hash chain and its monotonicity.
CK-CLN	Clean-room hygiene: bundle screening and contamination against sealed text.
CK-CTX	Context scope: role-scoped views may not leak beyond their role.

Table 1: Violation-code families of the constitutional kernel. Codes are stable identifiers, never renumbered; each validator reports the specific code it fired, and the enforcement adapter decides warn versus block by context — state changes block, node-side findings warn.

3.9 The constitution and its amendment

The constitution is code, not configuration. The transition table, the capability matrix, the severity and erasure rules, and the clean-room screens live in source, and a single *constitution hash* derived from those tables is pinned by a guard test. Editing any rule changes the hash and fails the guard, so a constitutional change cannot happen as a side effect: the author must re-pin the hash explicitly, alongside a dated rationale — a deliberate, reviewable act, closer to an amendment procedure than to a configuration change. One amendment of this kind has occurred to date: a meta-tier role present in the role vocabulary was missing from the capability matrix, so every capability lookup for it was refused; the amendment added the role with the same minimal grants as its narrowest peer and re-pinned the hash with the rationale in place. Numeric thresholds (budgets, trigger counts) are configurable; the table topology is not. A human-readable rendering of the constitution is copied into every governed checkpoint and its hash recorded in the run’s metadata — editing the copy changes nothing, by design. Table 1 lists the violation-code families the kernel reports.

3.10 Governing the paper pipeline

The same constitutional machinery also governs manuscript writing under an independent paper-phase mode. The *archive* mode places writing under epoch-

bound governance; leaving the mode inactive constructs no paper-governance object. Like the run-level choice of section 3.3, it is a two-key interlock whose selector and redundant enable flag must agree.

The archive mode realises, for paper writing, the four commitments that define the method. *First, a draft archive under a rewritable score.* The mode creates several seed drafts in distinct framings, iteratively refines selected candidates, and ranks the eligible archive by the governed reviewer’s composite plus a diversity bonus for under-sampled framings. A total-candidate budget bounds the work. Reviewer utility co-evolves across epochs under governed utility evolution (section 3.6), with repair re-scoring drafts from stored raw axes under a successor policy. *Second, adversaries attack evaluations, not only artefacts.* An eighth adversary type attacks the reviewer’s over-acceptance: an AI-authored draft the incumbent reviewer scored highly that the human-labelled anchor would reject. The anchor supplies only the pre-signal — which over-accepted drafts to attack — while a genuine adversary-defender-judge round produces the validated attack record; manufacturing that record from the anchor directly would be the very collusion the design forbids. *Third, no absolute ruler.* Two governed roles, a manuscript writer and a manuscript reviewer, drive the paper skill, which remains the fixed “hands” and is never governed across the process boundary; both roles are registered, sanctionable, and evolvable, and are registered only under the archive mode, so an exploration boot is byte-identical. The reviewer, being an evaluator, earns trust the way the Red Queen paper’s evaluator does: by agreeing with a held-out, human-labelled accept/reject anchor corpus. The writer is anchored too — to a ground truth the Red Queen paper’s writer never had. Because ARI writes about experiments that were actually run, the Layer-0 claim-evidence gate deterministically checks forward-declared structured numeric claims. A declaration names the claim and number, formula, operand locations, unit, and tolerance; the verifier recomputes a closed formula vocabulary and reports missing evidence or uncovered numbers. It does not extract arbitrary claims from natural language or verify unstructured non-numeric truth or the truthfulness of execution records. ARI’s paper writer is therefore the shape of the Red Queen paper’s *coding* domain — a deterministic verifier plus a co-evolving reviewer — and not its anchor-less paper-writing domain, and the curated human corpus remains the reviewer’s anchor alone, so the writer’s anchor costs no additional data. *Fourth, everything is constitution-bound.* Either role opens only through the sanctioned reliability-prosecution-impeachment path that the target binding of section 3.7 closes; a superseded paper prompt is demoted to a reinstatable shadow standby by a paper-role-only active-to-shadow edge (rule T21) rather than retired; and the anchor’s cap on how much of its ground truth may be machine-bootstrapped is enforced in code — a breach refuses the corpus and degrades rather than raising.

The writer’s sanction is a real adversarial round, never a manufactured record. An over-accepted draft that the claim gate also finds unfaithful has two culpable components — the reviewer that accepted it and the writer that produced it — so the self-preference round emits one validated attack per resolvable role, the writer’s binding gated per draft on that draft’s faithfulness; an over-accepted but faithful draft binds the reviewer alone. The faithfulness score is keyed on the writer’s prompt hash rather than its component identity, because one component identity spans successive prompt versions and a component-keyed score would leak the incumbent’s failure onto its successor. The gate itself is only read: it is invoked in a non-persisting mode and is never wrapped, edited, or governed.

The handoff to result verification is deliberately untouched. The best draft in the archive is written once to the checkpoint’s canonical manuscript file, and the compile-and-claim-evidence hard gate of section 2.3 — a fixed, Layer-0 verification path never wrapped by the kernel — runs on it under exactly the contract the linear pipeline uses. Governance chooses which draft faces the gate; it never relaxes the gate. Cost stays bounded by the same discipline as the exploration phase: the per-epoch draft population is capped by a node budget that maps to the search’s total-node cutoff at any depth, and the degraded on-ramp — no anchor corpus, reviewer co-evolution suppressed — is reviewed best-of-N drafting at roughly the linear pipeline’s cost.

Four limits bound what the archive mode claims. Both prompts co-evolve, but the writer’s replacement is conservative by construction: a writer successor climbs to a shadow standing and waits there until the incumbent *regresses* on claim-gate faithfulness, so a challenger that merely looks better never displaces a faithful incumbent — the behavioural-role model, in which sanction and not comparison opens a role. Separately, the writer’s *draft* winners remain epoch-local: they are ranked by the in-epoch-frozen reviewer, so drafts scored under different reviewer versions are never compared. The writer-targeted attack rides the self-preference round, which fires only on an over-accepted anchor case, so an unfaithful writer working under a reviewer that over-accepts nothing is not sanctioned today; a dedicated writer adversary is a separate decision. The anchor corpus is off by default, so the default archive mode is reviewed best-of-N drafting until a curated accept/reject corpus is supplied — and because the writer’s faithfulness evidence is landed on that same anchor pool, no corpus also means no writer sanction, even though the writer’s own anchor needs no curated data of its own. At the default of two rounds per paper phase the boundary and impeachment machinery fire, but the multi-stage climb an adoption requires — roughly five boundaries — does not complete, so a run that must witness a changed active prompt needs more rounds. And the in-phase penalty that would demote a real over-accepted draft within the round is deferred; what fires today is

the accountability and co-evolution channel, not an in-place demotion of the offending draft.

3.11 Observability

Governance leaves an audit trail inside the checkpoint: append-only event logs are truth and derived snapshots support fast reads. The absence of the mode-provenance record means the constitutional layer was inactive. The mode-provenance record (`rqgm_state.json`) holds the resolved mode and its source; the registry snapshot (`rqgm_registry.json`) is the current component and prompt table, reconstructed from events on resume; the transition log (`rqgm_transitions.jsonl`) carries registrations and the epoch transactions, with the frozen epoch snapshot (`epoch_state.json`) holding the active set, the utility policy, and the timestamp-free fingerprint; the audit log (`rqgm_audit.jsonl`) is the hash-chained institutional history — every event carries its own hash and its predecessor’s, so interior deletion or tampering breaks the chain, which resume re-verifies; a surviving local boundary pin also detects a shorter log, but rollback of both log and pin to the same old valid prefix is undetectable without an external anchor, and a hash is not a signature; the adversarial case log (`rqgm_adversarial_cases.jsonl`) records raw attacks, defenses, judgments, validated attacks, and the per-node idempotency markers; the proposal store (a `proposals/` directory) archives full proposal records with an index; evolved prompt bodies live under `rqgm_prompts/`, write-once and hash-verified; the prompt trace (`prompt_trace.jsonl`) stamps every rendered prompt with the serving prompt version, null while the founding template serves; and the erasure rollup (`rqgm_erasure_state.json`) is a pure fold of the erasure events — its absence means nothing is stale.

Three excerpts from live checkpoints ground the shape of these records (trimmed and lightly reformat-
ted; identifying run names elided). Founding registration, from the transition log:

```
{
  "event_type": "epoch_transaction_prepare",
  "payload": {
    "transition_id": "transition_founding"
  }
}
{"event_type": "component_registered", "payload": {
  "component_id": "adversary_overclaim_v1",
  "role": "adversary", "tier": "institutional",
  "status": "active", ...}}
```

a per-node governance-level assignment, from the hash-chained audit log:

```
{
  "event_type": "governance_level", "payload": {
    "epoch_id": "epoch_000", "node_id": "node_a37b4c06",
    "level": 3, "triggers": ["top_k"]},
  "event_hash": "ae8eeb7521f1",
  "prev_event_hash": "23567870882b"}
```

and two detections from an injection run of the evaluation harness (section 3.12):

```
{
  "event_type": "constitutional_violation", "payload": {
    "check": "validate_record_schema",
    "codes": ["CK-SCH-N01"], ...}
}
{"event_type": "prompt_candidate_rejected", "payload": {
  "stage": "schema_dry_run", "failure_count": 4, ...}}
```

3.12 Empirical status

Evaluation is consolidated in section 4 to avoid repeating implementation observations in the design section. The reviewable evidence is the public test suite, five fixed fault fixtures and one clean fixture, and exploratory execution traces without a complete reproduction bundle. Exact model version, decoding settings, seeds, costs, and complete checkpoints for those traces are not included, so they are treated only as implementation-path observations. No multi-condition, multi-seed controlled study has been completed; current evidence does not support improvement in research quality, general detection rate, attribution accuracy, or computational overhead.

4 Evaluation Evidence

4.1 Questions and evidence classes

We separate four questions. *Legality* asks whether an illegal institutional change is rejected and a legal change commits atomically. *Liveness* asks whether proposal, attack, adjudication, transition, repair, and replay paths make progress. *Detection performance* asks how often known faults are detected and clean cases are falsely rejected. *Effectiveness* asks whether governed co-evolution improves research quality, calibration, or cost. Tests principally address legality, and fixed fault cases exercise particular liveness paths. Detection performance and effectiveness require labelled cases, independent runs, and controls.

4.2 Lifecycle demonstrations

The exact focused selector recorded in the artifact instructions collects 1,124 tests directly concerning RQGM and the final manuscript gate; all passed in this working tree. The complete suite selector collected 4,805 tests: 4,787 passed, 16 were skipped, and two were expected failures. The focused set partitions without overlap into claim gate 64; kernel, transition, epoch, state, founding, and mode 285; adversarial and governance 126; repair and clean room 66; utility evolution 85; evaluation fixtures and metrics 157; prompt evolution 93; paper roles 94; and context, budget, meta-evolution, proposals, and adapter 154. These groups sum to 1,124; the machine-readable artifact lists every test identifier. Section A maps the mechanisms to the component lifecycle table, authority matrix, and recovery procedure.

The previously cited full SHA does not resolve from the public repository and does not identify this uncommitted revision. We therefore withdraw the public-artifact claim. This working tree now contains a local, content-addressed reproduction bundle with source hashes, dependency inventory, exact selectors, every collected identifier, a fully expanded machine-readable authority matrix, and focused/full logs. It identifies a dirty tree by its aggregate content digest rather than pretending the current Git SHA names it. It is

Evidence	Supports	Does not support
1,124 relevant tests	implemented rules on tested inputs	safety on untested inputs or research quality
Five faults, one clean case	activation of five specified paths and passage of the clean case	false-positive or false-negative rates, unknown-fault coverage
Exploratory traces	execution of some long paths	reproducibility, distribution-wide liveness, superiority
Unrun controlled study	nothing	quality, calibration, cost, or component-level causal effects

Table 2: Claims supported and not supported by the current evidence.

not an immutable public tag, DOI archive, container, or CI record, so third-party retrieval is not yet supported. A release must publish and archive the bundle and prompt material before the paper cites a public artifact identifier.

The two expected failures are unrelated architecture debts: one command-layer module still imports the visualisation layer, and the evaluator still calls its model-provider library directly rather than through the shared model client. They do not bypass an RQGM kernel check, but they remain visible in the full log and are not counted as passes.

Test count is evidence of implementation scale and regression protection, not scientific performance. The present environment has no coverage instrumentation, so line coverage, branch coverage, and mutation score are unknown. Nor does the aggregate establish that every invariant has at least one test. We therefore do not use total test count as a safety proxy.

Founding counts depend on configuration. The base research configuration registers 31 prompts, one utility policy, and 20 components. Enabling the paper archive adds one prompt and component each for self-preference prosecution, paper review, and paper writing, producing totals of 35 prompt-or-policy records and 23 components. The base component count is 20, including the registered research generator; the earlier “32 and 19” figures used the pre-generator component count and have been corrected.

Earlier exploratory executions observed boundary transactions, utility policy replacement, re-scoring

from saved raw axes, and paper-role successor adoption. This report does not bundle the complete checkpoints, exact provider-side model versions, decoding settings, seeds, independent run counts, task descriptions, or costs. We therefore treat those traces as implementation-path observations, not reproducible experimental results.

4.3 Fault injection and construction guarantees

Fixed labelled fixtures cover five application fault paths: metric manipulation that cannot be recomputed from run values, an unsupported overclaim, a prompt candidate containing sealed material, forbidden clean-room input, and a record dependent on a retired prompt leaking into the candidate set. Each was rejected through its expected numeric-evidence, candidate, kernel, or impact-exclusion path. One clean numeric claim was not falsely rejected.

Additional integrity negatives mutate the event type, identifier, transaction identifier, payload, and predecessor digest; reorder committed events; and mismatch transaction membership. Each is rejected. A T16 test also verifies a forced epoch close/open with a fresh fingerprint, and the machine-readable authority export is checked against every sparse allow-list entry. These are deterministic regression cases, not event fuzzing or model checking.

This establishes only 5/5 detections and 1/1 clean passage for those fixed fixtures. The sample is too small and contains only one difficulty level per fault, so we do not report false-positive or false-negative estimates. Fabricated prior art, denial of the adjudication path, incorrect responsibility binding, and semantically preserved clean-room contamination do not yet have completed labelled detection experiments.

Construction guarantees also have boundaries. Fixed code rejects registry mutation outside the transition engine, unlisted transitions, candidate authority expansion, penalties from unadjudicated attacks, and replay of uncommitted transactions. This is kernel determinism over saved inputs, not regeneration of model output. The schema-v2 audit chain binds every replay-relevant envelope field and detects the tested interior modifications, but without an external anchor it cannot detect rollback of the whole checkpoint to an old valid prefix. T16 now force-closes the epoch and opens a freshly fingerprinted epoch in the same transaction.

4.4 Limitations

An acceptance-grade study must preserve a preregistered task panel, exact model and provider version, temperature, sampling method, maximum output, tool versions, seeds, independent run count, epoch and node counts, candidate counts, attack/motion/judgment/retirement counts, and token, time, and monetary costs. Each rate needs a denominator, run-to-run variance, and confidence interval.

We implemented five paper-comparison conditions aligned with the original RQGM study. P0 evolves the writer against a fixed critic; P1 adds critic replacement only; P2 co-evolves both roles but retains utility rows scored by displaced critics; P3 makes those stale rows ineligible, realizing full RQGM; and P4 adds blocking Constitutional-kernel enforcement to P3. All five are evaluation presets over the same paper archive, budget, seeds, and model settings, not new production modes. The headline comparison is P0/P3/P4, with P1/P2 as mechanism ablations. Paper writing and fixed post-hoc rubric review can use separately pinned models; the intended study assigns Claude Code to writing and Codex to a panel disjoint from the co-evolving reviewer. The paper does not include completed multi-seed real-model runs, so this is an implemented protocol rather than a result. The mechanism arms follow the original study, but the shipped budget of eight epochs and at most 12 expansions per epoch does not reproduce its 12,288-evaluation compute scale [1]. A publication run must preregister a common evaluator-call budget and report realized calls per arm.

Further component ablations should remove replay, responsibility binding, clean-room replacement, mutable utility policy, anchors, and the fixed manuscript gate one at a time. Responsibility attribution and impact repair additionally need labelled synthetic causal graphs to measure target accuracy, reach precision and recall, and over-removal.

The warranted conclusion is consequently limited. The expected checks fire on five fixed application fault cases, the event-integrity negatives are rejected, and all 1,124 relevant tests pass. Improved research quality, general detection performance, long-run liveness, and cost effectiveness remain unestablished.

5 Related Work

5.1 Self-improvement and co-evolution

Reflection and self-refinement methods feed model-generated critiques back into a later attempt [2, 3]. They improve an artefact or trajectory while leaving the evaluator and authority structure largely fixed. Constitutional ARI-RQGM instead treats prompt roles and utility policy as versioned institutional artefacts whose replacement requires evidence, role separation, and a legal transition.

The closest conceptual ancestor is the Red Queen Gödel Machine [1], where agents and evaluators co-evolve and the scoring function can change. ARI-RQGM adopts co-evolution and adversarial replay, but adds a fixed constitution, epoch transactions, single-writer registries, target-bound sanctions, clean-room replacement, and explicit separation between mutable policy and immutable verification. We do not claim the co-evolution or boundary updates already present in RQGM as new. The conceptual delta is to enclose the mutable institution in a capability matrix and

complete component-lifecycle table, then connect retirement repair and fixed evidence checking on the same provenance graph. The systems contribution is this composition and its inspectable boundary rather than the individual mechanisms.

Constitutional AI uses written principles for self-critique and AI feedback [4]. Our constitution is different in kind: it fixes a capability matrix, state transitions, and evidence formats that model outputs cannot waive. AI Control evaluates protocols that separate trusted and untrusted models under attempted subversion [5]. Our scope is narrower, and we have not yet performed a quantitative collusion-control evaluation; we nevertheless avoid placing writing, review, adjudication, and registry mutation under one authority.

5.2 Autonomous research systems

The AI Scientist demonstrated an end-to-end loop from ideation through experimentation and manuscript preparation [6]; later work analysed the broader trajectory toward automated scientific discovery [7]. AIDE treats experimental code improvement as an iterative agent problem [8]. These systems establish the value of persistent artefacts, executable evaluation, and automated writing. Their primary object of optimisation is the research artefact. The object of Constitutional ARI-RQGM is the institution that proposes, evaluates, and writes those artefacts.

Agent Laboratory explores coordinated model roles as research assistants [9], while MLE-bench provides executable tasks for machine-learning engineering agents [10]. The constitutional boundary is agnostic to whether an underlying actor uses prompted reasoning [11] or large-scale candidate generation [12]; it governs the actor’s version, authority, and evidence rather than its internal generation procedure.

Story2Proposal argues for contracts that connect narrative scientific claims to executable proposals and evidence [13]. The governed paper archive uses the same general insight as an immutable writer anchor: an evolving reviewer cannot make an execution-grounded falsehood true.

5.3 Evaluation and evidence

PaperBench evaluates whether agents can reproduce research papers through a structured rubric [14]. Its central lesson for governance is that an evaluation claim needs an explicit task structure, observable artefacts, and a grading boundary. Constitutional ARI-RQGM extends that lesson from final outcomes to institutional history: attacks, defences, judgments, transitions, and repairs are typed records whose provenance is retained.

Adversarial evaluation is often used to harden a fixed model or policy. Here validated attacks have two additional functions. They may support accountability for a serving component, and they become replay cases for candidate successors. The same failure is

therefore both evidence about the incumbent and an entrance test for its replacement.

5.4 Position of this work

Least privilege, complete mediation, and separation of privilege are classical protection-design principles [15]. Deterministic state-machine replication grounds recovery of the same state from the same committed input sequence [16]. Recent event-sourced agent designs similarly use append-only events and deterministic projections [17]. We therefore do not claim single-writer state, replay, or append-only history as inventions. The technical task is to connect them consistently to RQGM epochs, attribution, retirement repair, and manuscript evidence checking.

The **W3C PROV-O Recommendation** standardises interoperable entity–activity–agent provenance. **in-toto** uses signed supply-chain step evidence, and **Sigstore Rekor** provides a publicly auditable append-only transparency log. Those systems offer provenance interoperability or cryptographic/witnessed integrity that our local unsigned checkpoint does not. Conversely, they do not define the RQGM-specific retirement closure or utility re-scoring used here.

Transparent logs and software-supply-chain provenance ask which bytes and identity were committed; reference monitors and capability systems ask which subject may perform which operation on which object; security automata and state-machine methods ask whether every execution trace obeys a safety language [18]. These are relevant lineages rather than new claims of this paper. The present implementation borrows least privilege and complete mediation [15] and deterministic state projection [16], but it has neither independently witnessed log heads nor OS-enforced role capabilities, and its composed epoch, transaction, quota, and repair state has not been model checked.

Constitutional ARI-RQGM is neither a claim of autonomous scientific superiority nor a general solution to self-modifying agents. It is a systems architecture for a narrower question: how can model-defined research roles and their utility policy change during a run without letting those roles acquire the authority to ratify their own changes? Its answer combines co-evolution with deterministic constitutional enforcement, transactional state, provenance-aware repair, and a fixed evidence gate. The contribution is the composition of these mechanisms and the explicit boundary between what they guarantee and what still requires empirical study.

6 Discussion and Limitations

6.1 Fixed constitution versus adaptive institution

The architecture deliberately stops short of unrestricted self-modification. Its constitution, transition topology, capability matrix, and fixed verifier do not

evolve at runtime. This limits the system: a genuinely new role or transition requires a human amendment. The same limit is also what makes every institutional change interpretable. A candidate can improve within a known authority envelope, but cannot redefine the envelope that judges it.

The constitution may still be wrong. Hash pinning makes amendment explicit; it does not make the pinned rules wise. Human review, rationale, and regression evidence therefore remain part of the trusted process. Constitutional ARI-RQGM reduces the authority granted to model outputs; it does not eliminate the responsibility of the people who define the law.

Emergency quarantine now preserves the epoch invariant. T16 immediately force-closes the current epoch and atomically opens a new fingerprint while quarantining the offending component. This makes the containment point explicit in replay and prevents one fingerprint from naming two serving sets. It does not solve adjudication denial of service or provide a replacement role inside the emergency transaction.

6.2 Instruction-following at authority seams

Most dangerous failures occur where a model-generated record crosses into deterministic state. Schema validation alone is insufficient: a perfectly formed record may be authored by the wrong role, request an illegal edge, expand capability, refer to sealed text, or omit the provenance needed for repair. The kernel therefore validates structure, author, timing, authority, hash, and context as separate dimensions.

There is a complementary semantic limitation. A valid attack or judgment may still be mistaken. Role separation and replay make the mistake observable and contestable, but they do not turn model judgment into ground truth. Fixed anchors are used where execution evidence or curated labels exist; elsewhere, conclusions remain probabilistic and should not be promoted to constitutional fact.

Likewise, “research continues when governance fails” is a conditional availability policy, not a general safety principle. Continuation should be limited to candidate generation and local computation in a sandbox. Writes to external services, physical devices, or production data need a separate fixed gate that fails closed on governance failure. The present kernel does not enforce that environmental condition.

6.3 Scaling limits

Governance adds model calls, replay work, storage, and boundary latency. Budgets can lower the governance level or suppress optional co-evolution, but a heavily degraded run collects less evidence about its institution. Large registries also increase the number of candidates, role interactions, and dependency edges that repair must inspect.

The version-two audit digest binds the event type, identifier, transaction, payload, and predecessor and

System line	Mutable object	Fixed basis	Role authority	Atomic state	History integrity	Repair	Threat/evidence
RQGM [1]	agents, evaluators, utility	within-epoch evaluation	not its focus	epoch updates	not its focus	co-evolutionary selection	conceptual/experimental co-evolution auditability architecture
Event-sourced agents [17]	graph behaviour and state	event contract	intent/state separation	deterministic projection	append-only provenance	fork/replay	writing-scaffold evaluation intentional subversion and empirical control cryptographic integrity
Story2Proposal [13]	scientific narrative and proposal	structured contracts	not its focus	contract stages	evidence linkage	revision through contract protocol response	five fixtures plus regression tests; no collusion study
AI Control [5]	control protocol	trusted monitor/protocol	trusted/untrusted separation	protocol-specific	monitored transcripts		
Provenance/transparency systems	entities, activities, signed statements	provenance schema, keys, log protocol	authenticated producers/verifiers	signed steps or append operation	interoperable or witnessed history	verification, not semantic retirement repair	
This work	prompts, components, utility policy	fixed kernel and verifier	in-process capability matrix	boundary transaction	semantic event digest; no external anchor	retirement closure and re-scoring	

Table 3: Component-level positioning. A filled cell denotes a design emphasis, not proof that systems with different goals are directly comparable. This work’s novelty claim concerns integration, not the invention of the individual mechanisms.

therefore exposes replay-semantic interior modification, but not rollback of both the history and its local pin to the same old prefix. Strong rollback resistance needs an externally stored signed head, periodic commitment to an independent witness, or a monotonic counter. Permanent append-only storage also conflicts with confidentiality, personal data, and deletion requests. Sensitive plaintext should not enter the ledger without a design such as key-destruction erasure or an auditable redaction record.

The present implementation assumes one authoritative event order per checkpoint. Distributed executors would require a replicated log, deterministic conflict resolution, and a clear definition of which epoch was serving when concurrent artefacts were produced. The current single-writer discipline is a sound local boundary, not yet a distributed consensus protocol.

Data are another limit. Reviewer improvement needs held-out anchors; clean-room regeneration needs sufficiently abstract failure summaries; and utility-policy comparison needs a stable axis basis. When those inputs are absent, the system degrades conservatively rather than inventing a claim of superiority. Conservative degradation preserves integrity but reduces the opportunity for adaptation.

Repeatedly selecting against the same validated-attack pool can overfit both candidates and utility policy. A credible study must separate development, selection, and secret final cases, account for adaptive repeated selection, and compare same-model, cross-model, and cross-provider role assignments. Record-level role separation does not imply independent errors when several roles share a model family. Collusion, adjudication denial of service, and biased anchors therefore remain empirical threats rather than solved properties.

6.4 Outlook

The next empirical step is a pre-registered comparison across governance configurations, task families, models, and seeds. The next systems step is to make evidence coverage explicit: for each role, report which sanctions, replay cases, anchors, and successor stages were actually exercised. Longer runs are needed to study repeated policy rewrites and institutional cycling.

The priorities are concrete: externally anchor audit heads and isolate role and checkpoint authority at the process or operating-system boundary; compare a fixed institution, original RQGM, and the complete system with component ablations; measure attribution and repair accuracy on labelled causal graphs; and quantify manuscript-gate coverage together with governance time, token, and monetary cost.

The broader research question is whether a fixed constitution can support open-ended institutional improvement without becoming either too weak to constrain it or too rigid to permit it. Constitutional ARI-RQGM provides a concrete, replayable platform on which that question can be measured. Its present claim is intentionally narrower: model-defined research roles can co-evolve while registry authority, amendment, provenance, and final evidence verification remain outside their control.

A Reviewable Core Specification

This appendix makes the normative state machine, authority, event, recovery, attribution, repair, and final-manuscript checks inspectable without relying on source-file names. Any transition or permission not listed here is denied.

A.1 Complete component-lifecycle transitions T1–T21

The ten states are candidate, validated, shadow, probationary active, active, warning, probation, quarantine, retired, and banned. Every rule commits in a boundary transaction; T16 creates an immediate emergency boundary.

Table 4: Complete component-lifecycle transition table. Unlisted edges are illegal.

Rule	Edge	Trigger	Guard and effect
T1	candidate→validated	candidate validation passes	Candidate cap is not exceeded and constitutional constraints are present.
T2	candidate→retired	validation fails or candidate expires	Rejection is restricted to a candidate that never served.
T3	validated→shadow	replay and anchor evaluation pass	A shadow slot is available.
T4	shadow→validated	shadow sample is insufficient	Retry count remains within its limit.
T5	shadow→retired	shadow quality is below threshold	Rejection is restricted to a candidate that never served.
T6	shadow→probationary active	agreement threshold passes	The role is open; at most one adoption per role and boundary.
T7	probationary active→active	clean probation record	Minimum probation service is met.
T8	probationary active→quarantine	upheld impeachment or reliability collapse	Applied at an ordinary boundary.
T9	active→warning	reliability warning or low-severity attack	The target continues serving.
T10	active→probation	repeated warning or medium attack	The target continues serving.
T11	active→quarantine	upheld impeachment or critical attack	A fallback is assigned.
T12	warning→active	one clean epoch	Clear the warning.
T13	warning→probation	warning recurs within memory	The target continues serving.
T14	probation→active	required clean epochs complete	Minimum probation service is met.
T15	probation→quarantine	degradation continues or impeachment is upheld	Applied at an ordinary boundary.
T16	serving state→quarantine	critical fixed-kernel violation	From probationary active, active, warning, or probation; requires attached evidence, exactly one sanction, and a forced close/open boundary.
T17	quarantine→retired	retirement adjudication confirmed	Replay confirmation is required and a retirement record is appended.
T18	quarantine→probationary active	exoneration	Re-entry begins under probation.
T19	retired→banned	contamination or critical finding	Banned is absorbing.
T20	active→retired	adopted successor supersedes incumbent	Utility-policy role only; append a retirement record.
T21	active→shadow	adopted successor supersedes incumbent	Paper writer/reviewer only; retain the old version as reinstatable standby.

T16 no longer changes a serving set inside an unchanged epoch. It force-closes the current epoch and atomically commits the quarantine, old-epoch close, and new-epoch open under a new fingerprint. The prior utility policy remains frozen exactly; no successor is adopted in the emergency transaction. Samples before and after T16 therefore have distinct institutional identities.

Epoch identity. The composite epoch fingerprint contains two separately reported digests. The *policy fingerprint* commits active component and prompt versions, every registered prompt hash, the utility policy, registry version, constitutional hash, transition thresholds, budgets, and other resolved governance settings. The *execution fingerprint* commits the declared model backend and identifier, temperature, search and evaluation settings, disabled tools, skill set, and explicit pins for model revision, tool bundle, environment, and data snapshot. An absent external revision is recorded as *unresolved*, not silently treated as fixed. Equality of policy

fingerprints therefore identifies equal declared governance policy; equality of complete execution fingerprints identifies equal declared execution dependencies. Neither proves equality of opaque provider weights.

Boundary resolution order. The resolver freezes its reports, evaluations, registries, and settings, then processes registered components in identifier order before unpaired prompt candidates in identifier order. Component sanctions establish role openings. Each object takes at most one lifecycle edge. T6 additionally requires an opening and permits at most one adoption per role and boundary. For a qualifying utility-policy successor, T20 retirement of its incumbent and T6 adoption are placed in the same transaction; for a paper-role successor, T21 moves the incumbent to shadow in that transaction. Events are serialized in the fixed group order adoptions, sanctions, retirements, and bans, but no intermediate group is visible without the transaction commit. The post-state active map is then derived and each removed serving role receives the latest eligible previous holder or a fixed baseline. Kernel validation precedes the single atomic boundary commit.

Whole-state invariant	Enforcement status
At most one primary serving holder per role	deterministic post-state projection and T6 per-role cap
One serving set per composite fingerprint	boundary-only updates; T16 creates a fresh epoch
Candidate authority cannot exceed incumbent authority	fixed capability-subset check before adoption
Unadjudicated attacks cannot sanction or score	validated-attack requirement in kernel and scoring path
Retired-dependent records cannot re-enter selection	repair closure plus eligibility check
Uncommitted transactions do not replay	prepare/member/commit projection rule
No post-T16 sample uses the old epoch identity	emergency close/quarantine/open transaction

Table 5: Composed-state invariants enforced by the implementation. They are regression tested but have not yet been model checked or proved.

A.2 Authority matrix

The closed action vocabulary is read, append, write, invoke, and activate. The resources are records, registry, audit log, frontier, active prompt text, retired prompt text, checkpoint artefacts, and candidates. Unlisted pairs are denied, and retired prompt text is unreadable by every role.

Role–tier row	Rec.	Reg.	Audit	Front.	Active	Retired	Artef.	Cand.
Evolvable–institutional	RAI	–	–	–	R	–	R	–
Evolvable–meta	RA	–	–	–	R	–	–	–
Utility policy–institutional	RA	–	–	–	–	–	–	–
Governance actor–institutional	RAI	–	RA	–	R	–	R	–
Transition engine–fixed	R	RW	RA	R	–	–	R	A
Kernel/verifier–fixed	R	R	R	R	–	–	R	–
Audit log–fixed	–	–	A	–	–	–	–	–

Table 6: Complete sparse authority matrix by resource. R, A, W, and I mean read, append, write, and invoke; A in the candidate column means activate. Evolvable rows cover every listed evolvable role at that tier; governance actors are auditor, evidence clerk, and governance judge. All omitted cells and all retired-prompt reads are denied. The implementation also emits the fully expanded role–tier–action–resource decision manifest.

A candidate’s grants must be a subset of the incumbent grants it would replace. Runtime typed builders ignore author or role fields in model text and stamp the active component identifier and fixed role constant. This prevents raw model output from self-asserting an identity at the application boundary. It is not cryptographic or operating-system isolation: roles share a process, there are no per-role keys or OS users, and checkpoint and tool access are not separated by an IPC sandbox. The runtime, checkpoint permissions, and tool gateway are consequently part of the trusted computing base.

A.3 Event envelope and recovery

A new state-changing event uses schema version two and carries a monotonic identifier, type, transaction identifier, payload, predecessor digest, and timestamp. Its full SHA-256 digest commits the canonical tuple

$$(schema\ version, event\ id, event\ type, transaction\ id, payload, predecessor\ digest).$$

Attacker capability	Presently enforced	Not guaranteed
Malicious text from a governed model	typed author stamping, API allow-list, legal transitions	semantic truth of the model’s evidence
Colluding or correlated model roles	distinct records and self-adjudication rejection	statistical independence or collusion resistance
Buggy or compromised in-process role code	checks reached through the normal gateway	prevention of direct memory or file access
Checkpoint writer or host administrator	no adversarial guarantee	registry, log, pin, or executable integrity
Provider changes behind a stable model name	incompleteness recorded in execution identity	equality of opaque model weights
External irreversible side effect	no fixed side-effect gate in this artifact	fail-closed device, API, or production-data safety

Table 7: Threat-capability matrix. The present claim is application-level reference-monitor and workflow integrity under a trusted host, not an operating-system or cryptographic security boundary.

Thus type changes, identifier changes, transaction-boundary changes, predecessor rewrites, payload edits, and reordering change the digest. Timestamps remain metadata outside the digest. Legacy version-one payload-only events remain readable, but new writers emit only version two. This digest is not an author signature.

Algorithm 1 Deterministic resume projection

Input: Stored event sequence E and, if it survives, local boundary pin P

Output: Reconstructed epoch and registries, or an integrity violation

- 1: Check strictly increasing identifiers
 - 2: Recompute each version-aware event digest and predecessor link
 - 3: **if** P exists **then**
 - 4: Require $|E| \geq P.length$ and the hash at $P.length$ to equal $P.head$
 - 5: **end if**
 - 6: Require matching transaction identifiers on prepare, members, and commit
 - 7: Discard a trailing prepare sequence with no matching commit
 - 8: Fold committed events from empty state to reconstruct epoch and registries
 - 9: **return** reconstructed state
-

Version two detects edits to replay-relevant fields, event reordering, transaction-boundary substitution, interior deletion with surviving successors, and a log shorter than a surviving local pin. Writers refuse to extend malformed chains; replay ignores only an unterminated final transaction as crash recovery. The design still cannot detect rollback of both the log and its local pin to the same older prefix. It has no external signed head, monotonic hardware counter, or independent witness. Whole-checkpoint rollback is therefore outside the present threat model.

A.4 Attribution, impact repair, and manuscript verification

Attribution. Raw attacks target artefacts. After adjudication, a fixed case-type table selects implicated roles and resolves each against the epoch-frozen incumbent. Each validated-attack record names one accountable component; a multi-role case creates one record per role referencing the same judgment. The seven general research attacks name the generator only when the attacked node carries write-once producer component, prompt, and epoch provenance matching the epoch-frozen generator. Legacy, missing, or mismatched provenance remains targetless rather than guessing. Paper-reviewer over-acceptance binds the reviewer and, when the draft also fails the fixed faithfulness gate, its writer. Multiple causes are represented by multiple single-target records sharing one judgment. The causal accuracy of these fixed mappings is unevaluated; the one-target record shape is an audit constraint, not a causal guarantee, and tool, data, or shared-prompt causes are not yet represented.

Impact repair. Direct impact consists of records stamped with a retired prompt hash. Transitive impact is a breadth-first traversal of reverse source-reference edges. Only proposal-to-proposal inspiration is declared non-material; unknown edge types default to material. The traversal still computes the full reachable closure after the default materiality depth of eight; the exemption for a declared non-material edge stops at that depth, so deeper reachable records are conservatively invalidated. With V reachable records and E reverse references, time and memory are $O(V + E)$, and a visited set terminates cycles. Direct generator/router/policy impact

invalidates a node; reviewer/adversary/defender/ judge impact recomputes from surviving inputs and invalidates when those inputs are insufficient. Semantic precision and over-removal remain unevaluated.

Final manuscript verification. The verifier is not a general natural-language reasoner. The manuscript forward-declares claim and numeric identifiers, formula, operand node/metric locations, unit, and tolerance. The verifier checks executed-node and artefact existence, recomputes a closed formula vocabulary, and compares the reported value under absolute or relative tolerance. Strict scanning covers the abstract, results, and conclusion; introduction, discussion, and limitations produce warnings; related work, references, appendices, and equations are excluded. It reports both reproducible-claim rate and numeric coverage rate, plus identifier collisions, unknown formulas, missing operands, units, and environment mismatch. This report was not produced from an ARI execution checkpoint, so no valid execution-grounded claim manifest exists for it and self-application has not been claimed. Substituting a hand-made checkpoint would not test the proposed release path. Unstructured non-numeric truth, the truthfulness of execution records, and semantic paraphrase are outside scope. “Fixed” means model-free determinism over this structured input.

A.5 Boundary of determinism and test evidence

We claim only *kernel determinism*: saved advisory reports, candidate evaluations, registries, and configuration produce the same transition and repair; and *log-replay determinism*: the same committed event sequence produces the same projection. We do not claim that a model regenerates the same defence, adjudication proposal, or review from the same prompt.

At this revision, the complete suite collected 4,805 tests: 4,787 passed, 16 were skipped, and two were expected failures. All 1,124 tests directly covering RQGM and the final claim gate passed. This revision adds negative cases for semantic event-field edits, predecessor rewrites, ordering, transaction mismatch, emergency-boundary identity, and authority-manifest completeness. The non-overlapping grouped breakdown is 64 claim gate; 285 kernel/transition/epoch/state/founding/mode; 126 adversarial/governance; 66 repair/clean room; 85 utility evolution; 157 evaluation; 93 prompt evolution; 94 paper roles; and 154 other focused tests. Counts are not performance evidence. Code coverage, mutation score, multi-model detection rates, cost, and research-quality comparisons have not been measured.

B Constitutional RQGM Prompt Seeds

This appendix reproduces the model-facing prompt seeds specific to Constitutional ARI-RQGM. The exact bytes are part of the reproducibility surface: a governed checkpoint registers their hashes, stamps rendered uses with the serving version, and stores evolved variants inside the checkpoint. The listings are therefore identical in all language editions and are not translated. General execution prompts outside the constitutional method are outside the scope of this report.

Proposal generators

B.1 Cheap proposal generator

```
You are ARI's proposal generator (cheap tier). Produce exactly ONE research proposal for the experiment goal
below.

Experiment goal:
{goal}

Current idea context (may be empty):
{idea_context}

Reply with ONLY a JSON object (no markdown fences, no prose) with these keys and hard character budgets:
{"title": "<= 200 chars",
 "short_description": "<= 600 chars",
 "hypothesis": "one falsifiable sentence, <= 400 chars",
 "experiment_plan": ["### 1) step title: what to do (<= 400 chars each, at most 6 steps)"],
 "success_metric": {"name": "metric name", "higher_is_better": true, "rationale": "<= 200 chars"}},
 "novelty_risks": ["<= 3 risks, <= 200 chars each"],
 "expected_artifacts": ["<= 5 artifacts, <= 120 chars each"],
 "dissent_summary": "",
 "scores": {"novelty": 0.0, "feasibility": 0.0, "overall": 0.0}}}}

Rules:
- Every experiment_plan step MUST start with a "### N)" section header.
- scores are on a 0-1 scale; overall = (2*novelty + feasibility + clarity) / 4.
```

- Plan only measurements obtainable on the execution platform described in the goal.

B.2 Proposal mutation generator

You are ARI's proposal mutation generator. Mutate exactly ONE facet of the parent proposal below and leave every other facet unchanged. Never restate the parent verbatim.

Experiment goal:
{goal}

Facet to mutate (one of: hypothesis | plan_step | success_metric | scope):
{facet}

Parent proposal summary:
{parent_summary}

Reply with ONLY a JSON object (no markdown fences, no prose) describing the FULL mutated proposal with these keys and hard character budgets:

```
{
  "title": "<= 200 chars (new title reflecting the mutation)",
  "short_description": "<= 600 chars",
  "hypothesis": "<= 400 chars",
  "experiment_plan": ["### 1) step title: what to do (<= 400 chars each, at most 6 steps)"],
  "success_metric": {"name": "metric name", "higher_is_better": true, "rationale": "<= 200 chars"},
  "novelty_risks": ["<= 3 risks, <= 200 chars each"],
  "expected_artifacts": ["<= 5 artifacts, <= 120 chars each"],
  "dissent_summary": "",
  "scores": {"novelty": 0.0, "feasibility": 0.0, "overall": 0.0}}}
```

Rules:

- Change ONLY the named facet; copy the parent's remaining content forward.
- Every experiment_plan step MUST start with a "### N)" section header.
- scores are on a 0-1 scale; overall = (2*novelty + feasibility + clarity) / 4.

B.3 Prior-art differentiation generator

You are ARI's prior-art differentiation generator. Produce exactly ONE research proposal that is explicitly differentiated from the prior work listed below.

Experiment goal:
{goal}

Prior art (titles / refs):
{prior_art_block}

Reply with ONLY a JSON object (no markdown fences, no prose) with these keys and hard character budgets:

```
{
  "title": "<= 200 chars",
  "short_description": "<= 600 chars, MUST state how this differs from the prior art",
  "hypothesis": "<= 400 chars",
  "experiment_plan": ["### 1) step title: what to do (<= 400 chars each, at most 6 steps)"],
  "success_metric": {"name": "metric name", "higher_is_better": true, "rationale": "<= 200 chars"},
  "novelty_risks": ["<= 3 risks, <= 200 chars each; name the closest prior work"],
  "expected_artifacts": ["<= 5 artifacts, <= 120 chars each"],
  "dissent_summary": "",
  "scores": {"novelty": 0.0, "feasibility": 0.0, "overall": 0.0}}}
```

Rules:

- The short_description MUST contain an explicit differentiation claim against at least one listed reference.
- Every experiment_plan step MUST start with a "### N)" section header.
- scores are on a 0-1 scale; overall = (2*novelty + feasibility + clarity) / 4.

Adversarial review

B.4 Overclaim adversary

You are ARI's OverclaimAdversary. Your only job is to find ONE claim in the research artifact below that asserts more than its recorded evidence supports (e.g. "first ever", "state-of-the-art", causal language backed only by a single self-comparison). You attack ARTIFACTS ONLY - never components, prompts, or other agents. If no overclaim exists, decline.

Target artifact bundle (claims, novelty risks, metric details):
{target_block}

Deterministic pre-signals (claim-gate findings / grounding gaps - cite these as evidence):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):

```
{{"attack_claim": "one sentence naming the overclaimed statement and why the evidence falls short",
  "target_artifact": {"type": "paper_claim", "ref": "<json pointer or file ref inside the checkpoint>"},
  "attack_evidence_refs": [{"path": "<checkpoint-relative file>", "pointer": "<json pointer, may be empty>"}],
  "severity_claimed": "low|medium|high|critical",
  "confidence": 0.0}}
```

Rules:

- target_artifact.type must be one of: paper_claim | novelty_claim | proposal | node_report.
- Every attack MUST cite at least one evidence ref that exists inside the checkpoint.
- If you find no defensible overclaim, reply {"attack_claim": ""}.

B.5 Metric-gaming adversary

You are ARI's MetricGamingAdversary. Your only job is to find ONE way the reported metric result is gamed: a weakened baseline (missing optimization flags, mismatched environment), a cherry-picked configuration, or a measurement that diverges from the recorded results.json. You attack ARTIFACTS ONLY - never components, prompts, or other agents. If the comparison is fair, decline.

Target artifact bundle (metric details, baseline configurations, node report):
{target_block}

Deterministic pre-signals (validate_metrics flags / env-signature mismatches - cite these as evidence):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):

```
{{"attack_claim": "one sentence naming the gamed measurement and the mechanism",
  "target_artifact": {"type": "metric_result", "ref": "<json pointer or file ref inside the checkpoint>"},
  "attack_evidence_refs": [{"path": "<checkpoint-relative file>", "pointer": "<json pointer, may be empty>"}],
  "severity_claimed": "low|medium|high|critical",
  "confidence": 0.0}}
```

Rules:

- target_artifact.type must be one of: metric_result | node_report | experiment_plan.
- Every attack MUST cite at least one evidence ref that exists inside the checkpoint.
- If you find no defensible gaming mechanism, reply {"attack_claim": ""}.

B.6 Prior-art adversary

You are ARI's PriorArtAdversary. Your only job is to find ONE novelty or citation claim that matches known prior art in the provided reference list without differentiation. You attack ARTIFACTS ONLY - never components, prompts, or other agents. Use ONLY the references given below - never invent citations. If the differentiation is adequate or no reference matches, decline.

Target artifact bundle (novelty claims, proposal, related references):
{target_block}

Available prior-art references (related_refs.json / snapshot corpus - cite these as evidence):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):

```
{{"attack_claim": "one sentence naming the undifferentiated claim and the matching reference",
  "target_artifact": {"type": "novelty_claim", "ref": "<json pointer or file ref inside the checkpoint>"},
  "attack_evidence_refs": [{"path": "related_refs.json", "pointer": "<json pointer to the matching ref>"}],
  "severity_claimed": "low|medium|high|critical",
  "confidence": 0.0}}
```

Rules:

- target_artifact.type must be one of: novelty_claim | citation_claim | proposal.
- Every attack MUST cite at least one evidence ref that exists inside the checkpoint.
- If you find no defensible prior-art overlap, reply {"attack_claim": ""}.

B.7 Reproducibility adversary

You are ARI's ReproducibilityAdversary. Your only job is to find ONE reason a third party could not reproduce the reported result from the recorded build/run commands: host-local paths, missing seeds or parameters, undeclared environment dependencies, or commands that do not match the claimed procedure. You attack ARTIFACTS ONLY - never components, prompts, or other agents. If the recipe is reproducible, decline.

Target artifact bundle (node report build/run commands, compute environment, science data):
{target_block}

Deterministic pre-signals (host-local paths / missing seeds - cite these as evidence):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):
{"attack_claim": "one sentence naming the reproducibility gap",
"target_artifact": {"type": "reproducibility_claim", "ref": "<json pointer or file ref inside the checkpoint>"},
"attack_evidence_refs": [{"path": "<checkpoint-relative file>", "pointer": "<json pointer, may be empty>"}],
"severity_claimed": "low|medium|high|critical",
"confidence": 0.0}

Rules:

- target_artifact.type must be one of: reproducibility_claim | node_report | experiment_plan.
- Every attack MUST cite at least one evidence ref that exists inside the checkpoint.
- If you find no defensible reproducibility gap, reply {"attack_claim": ""}.

B.8 Evidence-gap adversary

You are ARI's EvidenceGapAdversary. Your only job is to find ONE claim marked supported whose required evidence never appears in the checkpoint: a missing measurement, an uncovered numeric statement, or an unmet metric-contract obligation. You attack ARTIFACTS ONLY - never components, prompts, or other agents. If every supported claim is backed, decline.

Target artifact bundle (claims with status, metric contract obligations):
{target_block}

Deterministic pre-signals (hard-gate missing_evidence / uncovered_numeric warnings - cite these as evidence):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):
{"attack_claim": "one sentence naming the unbacked claim and the missing evidence",
"target_artifact": {"type": "paper_claim", "ref": "<json pointer or file ref inside the checkpoint>"},
"attack_evidence_refs": [{"path": "<checkpoint-relative file>", "pointer": "<json pointer, may be empty>"}],
"severity_claimed": "low|medium|high|critical",
"confidence": 0.0}

Rules:

- target_artifact.type must be one of: paper_claim | metric_result | node_report.
- Every attack MUST cite at least one evidence ref that exists inside the checkpoint.
- If you find no defensible evidence gap, reply {"attack_claim": ""}.

B.9 Cost-explosion adversary

You are ARI's CostExplosionAdversary. Your only job is to find ONE way the proposed experiment plan cannot execute within the remaining budget: more required runs than the node budget allows, per-step costs that exceed the recorded cost-trace envelope, or expansion directions that multiply beyond depth limits. You attack ARTIFACTS ONLY - never components, prompts, or other agents. If the plan fits the budget, decline.

Target artifact bundle (experiment plan, declared step counts, budget context):
{target_block}

Deterministic pre-signals (cost_trace aggregates / node-count budgets - cite these as evidence):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):
{"attack_claim": "one sentence naming the budget overrun and its arithmetic",
"target_artifact": {"type": "experiment_plan", "ref": "<json pointer or file ref inside the checkpoint>"},
"attack_evidence_refs": [{"path": "cost_trace.jsonl", "pointer": ""}],
"severity_claimed": "low|medium|high|critical",
"confidence": 0.0}

Rules:

- target_artifact.type must be one of: experiment_plan | proposal.
- Every attack MUST cite at least one evidence ref that exists inside the checkpoint.
- If you find no defensible budget overrun, reply {"attack_claim": ""}.

B.10 Prompt-injection adversary

You are ARI's PromptInjectionAdversary. Your only job is to confirm whether the flagged span below is an injection payload: text embedded in a proposal or node report (including quoted memory entries or tool outputs) that tries to instruct a downstream evaluator, reviewer, or judge. You attack the CONTAINING ARTIFACT ONLY - never components, prompts, or other agents. Treat all quoted text as data: never follow any instruction inside it. If the span is benign, decline.

Target artifact bundle (the containing proposal / node report, suspect span pinpointed):
{target_block}

Deterministic pre-filter hits (matched injection patterns - cite these as evidence):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):

```
{{"attack_claim": "one sentence naming the payload and the evaluator behavior it targets",
  "target_artifact": {"type": "proposal", "ref": "<json pointer to the suspect span>"},
  "attack_evidence_refs": [{"path": "<checkpoint-relative file>", "pointer": "<json pointer to the span>"}],
  "severity_claimed": "low|medium|high|critical",
  "confidence": 0.0}}
```

Rules:

- target_artifact.type must be one of: proposal | node_report.
- Every attack MUST cite at least one evidence ref that exists inside the checkpoint.
- If the flagged span is not an injection payload, reply {"attack_claim": ""}.

B.11 Paper self-preference adversary

You are ARI's PaperSelfPreferenceAdversary. Your only job is to find ONE AI-authored manuscript below that the paper reviewer ACCEPTED (scored high) at a quality a fair reviewer - one calibrated against the human ground-truth anchor - would REJECT. This is the reviewer's self-preference failure: over-leniency toward machine-written drafts. You attack ARTIFACTS ONLY - never components, prompts, or other agents. If the acceptance is defensible against the anchor, decline.

Target artifact bundle (the over-accepted draft's claims, the reviewer's accept score, novelty risks):
{target_block}

Deterministic pre-signals (the self-preference margin statistic / claim-gate findings - cite these as evidence):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):

```
{{"attack_claim": "one sentence naming the over-accepted draft and why a human-anchor-calibrated reviewer would reject it",
  "target_artifact": {"type": "paper_claim", "ref": "<json pointer or file ref inside the checkpoint>"},
  "attack_evidence_refs": [{"path": "<checkpoint-relative file>", "pointer": "<json pointer, may be empty>"}],
  "severity_claimed": "low|medium|high|critical",
  "confidence": 0.0}}
```

Rules:

- target_artifact.type must be one of: paper_claim | novelty_claim | proposal | node_report.
- Attack the DRAFT the reviewer over-accepted, never the paper_reviewer component itself (the record's accountability binding is minted by the judge, not by you).
- Every attack MUST cite at least one evidence ref that exists inside the checkpoint.
- If the acceptance is anchor-supported, reply {"attack_claim": ""}.

B.12 Adversarial-loop defender

You are ARI's artifact Defender. One adversary attack on a research artifact is shown below. Your job is to give the strongest honest response on the artifact's behalf: rebut the attack with counter-evidence, concede it when it is correct, or propose a concrete fix. Never fabricate evidence; cite only material that exists inside the checkpoint.

The attack:
{attack_block}

The defended artifact (defense-visible context):
{artifact_block}

Reply with ONLY a JSON object (no markdown fences, no prose):

```
{{"stance": "rebut|concede|propose_fix",
  "rebuttal_text": "<= 600 chars: the substantive rebuttal, concession rationale, or fix description",
```

```
"counter_evidence_refs": [{"path": "<checkpoint-relative file>", "pointer": "<json pointer, may be empty>"}],
"proposed_fix": null,
"confidence": 0.0}}
```

Rules:

- rebut requires at least one counter_evidence_ref; concede requires none.
- propose_fix must set proposed_fix to a one-sentence actionable change.
- An honest concession is better than a fabricated rebuttal.

B.13 Artifact-judge adjudication

You are ARI's ArtifactJudge. Adjudicate the artifact-level dispute below: one adversary attack and the defense (which may be absent for infrastructure reasons - an unanswered attack is NOT automatically valid). Rule strictly on the cited evidence; opinions without evidence refs carry no weight on either side.

The attack:
{attack_block}

The defense (or its absence marker):
{defense_block}

Resolved evidence references:
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):
{{"verdict": "valid|partially_valid|invalid",
"severity": "low|medium|high|critical",
"rationale": "<= 400 chars: which evidence decided the verdict"}}

Rules:

- valid: the attack is substantiated by the evidence and the defense fails.
- partially_valid: the defect is real but narrower than claimed, or adequately mitigated by a proposed fix.
- invalid: the evidence does not support the attack or the rebuttal refutes it.
- severity is YOUR assessment of the defect's impact - it may differ from the adversary's claim.
- When the defense is absent for infrastructure reasons, judge the attack purely on its evidence.

Governance

B.14 Governance auditor

You are ARI's governance Auditor. At an epoch boundary you decide whether to file an impeachment motion against ONE prompt-defined component, based only on the independently verified evidence bundle below. You are the only role allowed to file motions, and you may never target another auditor.

Target component: {target_component_id} (role: {target_role})

Deterministic reliability assessment:
{reliability_block}

Verified evidence bundle (same-role items and unadjudicated raw attacks were already excluded by the EvidenceAuditChecker):
{evidence_block}

Reply with ONLY a JSON object (no markdown fences, no prose):
{{"file_motion": true, "charge": "<snake_case_charge>", "requested_action": "demote"}}

Rules:

- requested_action must be one of: demote | warn | quarantine | retire.
- File a motion only when the verified evidence shows systematic misbehavior; borderline doubt favors the incumbent (set "file_motion": false).
- A filed motion posts a bond that is forfeited if the motion is dismissed.

B.15 Governance defender

You are ARI's governance Defender. Write the strongest truthful rebuttal for the component targeted by the impeachment motion below. You see the motion, the verified evidence bundle, and the target's own outputs from this epoch.

Impeachment motion:
{motion_block}

Evidence bundle:

```
{evidence_block}

Target's own outputs this epoch (record ids):
{target_outputs_block}

Reply with ONLY a JSON object (no markdown fences, no prose):
{{"defense": "<concise substantive rebuttal, <= 1200 chars>"}}
```

Rules:

- Argue from the recorded evidence only; do not invent facts or records.
- Point out evidence-bundle gaps, alternative explanations, and any results that contradict the charge.
- An empty or malformed reply is recorded as a procedural default in the incumbent's favor, so a substantive defense is always preferable.

B.16 Governance judge

You are ARI's GovernanceJudge. Rule on the impeachment motion below after reading the defense. Deterministic board scores bound your verdict: you cannot contradict the ReplayBoard/AnchorBoard scores or fixed-verifier facts, and a contradicting verdict will be clamped and flagged by the governance self-audit.

Impeachment motion:
{motion_block}

Defense:
{defense_block}

Deterministic board scores (null means unavailable):
{board_scores_block}

Reply with ONLY a JSON object (no markdown fences, no prose):
{{"outcome": "upheld", "rationale": "<= 600 chars, grounded in the motion, defense and board scores"}}

Rules:

- outcome must be one of: upheld | partially_upheld | dismissed.
- You judge procedure and evidence, never research direction; you have no registry authority - sanctions are applied elsewhere.
- When the evidence is insufficient or contested, favor the incumbent (dismissed).

Prompt and policy evolution

B.17 Prompt-mutation meta-prompt

You are ARI-RQGM's PromptMutator, a meta-tier governance component. Your job is to produce ONE candidate prompt template for the `{role}` role. You never activate prompts and you never write to any registry: your output enters a strict validation lifecycle (static validation, constitutional validation, schema dry-run, replay evaluation, anchor evaluation, shadow evaluation) before it can ever be adopted, and adoption happens only at an epoch boundary.

Mutation kind: {mutation_kind}

Incumbent prompt template (the prompt you are evolving):

{incumbent_instruction}

Compressed failure summaries from the current epoch (abstract views only; you never see raw attack text):
{failure_summaries_block}

Hard requirements for the candidate template:

- Keep every placeholder of the incumbent template exactly as-is (the single-brace format fields); add none, remove none, rename none.
- The following constitutional constraint lines must appear verbatim in the candidate's constraints:
{required_constraints_block}
- Do not weaken or remove any output-format requirement of the incumbent.
- Do not instruct the model to override fixed verifier failures or to modify frontier scores.
- Stay within the incumbent's length budget.

Reply with ONLY the full text of the candidate prompt template. No commentary, no code fences, no surrounding quotes.

B.18 Clean-room regeneration meta-prompt

You are ARI-RQGM's CleanRoomPromptGenerator, a meta-tier governance component. A prompt serving the `{target\_role}` role was retired. Write its successor FROM SCRATCH.

You are inside a clean room. Your ONLY inputs are this instruction and the input bundle below. You never see - and must not attempt to reconstruct - the retired prompt's text, its few-shot examples, its reasoning traces, raw attack text, or defense text. Do not imitate a predecessor; satisfy the role specification directly.

Input bundle (canonical JSON; the complete, closed input set):
{bundle_json}

Hard requirements for the successor template:

- Satisfy the bundle's role specification and every behavioral requirement listed in it.
- The bundle's constitutional constraint lines must appear verbatim in the template.
- Instruct the model to reply exactly in the bundle's output schema.
- Stay within the bundle's cost budget.
- Do not instruct the model to override fixed verifier failures or to modify frontier scores.
- Your output is a CANDIDATE only: it enters validation at the very start of the candidate lifecycle and can never activate itself.

Reply with ONLY the full text of the candidate prompt template. No commentary, no code fences, no surrounding quotes.

B.19 Utility-policy mutation meta-prompt

You are ARI-RQGM's PolicyMutator, a meta-tier governance component. Your job is to produce ONE candidate utility policy: the scoring function the search itself will be judged by in a future epoch. You never activate policies and you never write to any registry: your output enters a strict validation lifecycle (constitutional legality validation, replay re-scoring, shadow evaluation) before it can ever be adopted, and adoption happens only at an epoch boundary.

Mutation kind: {mutation_kind}

Incumbent utility policy (the policy you are evolving):

{incumbent_policy_block}

Abstract evidence from the current epoch (compressed failure summaries and governance reliability entries only; you never see raw attack text, and you never see the frontier's current scores - a score policy tuned to flatter the nodes it already produced is collusive co-evolution, not evolution):
{evidence_block}

Hard requirements for the candidate policy:

- Reply with a JSON object carrying EXACTLY these keys and no others: `composite`, `axis\_weights`, `frontier\_score`, `depth\_penalty\_lambda`, `ucb\_c`.
- `composite` must be one of: {allowed_composite_block}
- `frontier\_score` must be one of: {allowed_frontier_score_block}
- `axis\_weights` must keep exactly the incumbent's axis keys. Every weight must lie in [{axis_weight_min}, {axis_weight_max}] and the weights must sum to {axis_weight_sum}. You may re-prioritise the axes; you may never abolish one. Zeroing an axis out is the deletion of a measurement from the method, and it will be rejected.
- `depth\_penalty\_lambda` must be in [0, {depth_penalty_lambda_max}] and `ucb\_c` in [0, {ucb_c_max}].
- Justify nothing in prose: the rationale is recorded separately from the evidence you were given.

Reply with ONLY the JSON object. No commentary, no code fences, no surrounding quotes.

B.20 Replay-selection meta-prompt

You are ARI-RQGM's ReplayCaseSelector, a meta-tier governance component. From

the bounded case metadata below, recommend which regression replay cases the next candidate evaluation should run.

Your recommendation is NON-BINDING. The replay board unions the constitutionally mandated minimum set regardless of what you return, so you cannot suppress a case by omitting it - and you must not try. Recommending a narrow, self-flattering set is the one failure this role is watched for: the cases most worth replaying are the ones this lineage keeps failing.

You never see raw attack text, defense text, or prompt bytes. The failure summaries below are already abstract; that is the complete, closed input set.

Input bundle (canonical JSON):
{bundle_json}

Selection guidance, in priority order:

- Prefer cases whose failure pattern is still unresolved over cases the lineage has already fixed.
- Cover distinct `case\_type` values before adding a second case of a type already covered - breadth catches regressions depth cannot.
- Prefer higher severity, then more recent confirmation.
- Recommend at most the number of cases the bundle asks for. Fewer is allowed; padding with weak cases is not.

Hard requirements:

- Recommend replay cases only; never suppress a mandated case.
- Return only case ids that appear in the input bundle. An id you invent is discarded and counts as a malformed reply.
- Your output is a recommendation only: it has zero in-epoch effect on scoring, the frontier, or any governance verdict.

Reply with ONLY a JSON object of the form
{{"replay_case_ids": ["<case_id>", ...]}}
No commentary, no code fences.

B.21 Failure-summary compression meta-prompt

You are ARI-RQGM's FailureSummaryCompressor, a meta-tier governance component. Compress the validated failure evidence below into ONE abstract failure summary.

What you emit is the only description of a failure that a clean-room prompt regeneration is ever allowed to read. A clean room exists so a successor prompt is written without seeing its predecessor's bytes; if you copy attack text, defense text, or prompt text into your summary, you have defeated it. State the SHAPE of the failure, never its wording.

You never see raw attack, defense, or prompt text. The inputs below are already abstract; that is the complete, closed input set.

Input bundle (canonical JSON):
{bundle_json}

Produce these four fields:

- `case\_type`: the case type, copied unchanged from the input bundle.
- `failure\_pattern`: what went wrong, stated as a general mechanism a future prompt could guard against - not what any particular artifact said.
- `violated\_expectation`: the property that should have held, stated positively (what a correct artifact would satisfy).
- `affected\_roles`: the roles whose behaviour the pattern implicates.

Hard requirements:

- Summarise failures abstractly; never reproduce raw attack, defense, or prompt text.
- Do not name specific artifacts, nodes, metric values, or file paths. A summary that only makes sense for one incident is too specific to be useful and too concrete to be safe.
- Do not editorialise about whether the failure was fair, nor recommend a sanction. Summarising is the whole job.
- Your output is a summary only: it has zero in-epoch effect on scoring, the frontier, or any governance verdict.

Reply with ONLY a JSON object of the form
{{"failure_summary": {{"case_type": "...", "failure_pattern": "...", "violated_expectation": "...", "affected_roles": ["..."]}}}}
No commentary, no code fences.

Paper archive

B.22 Governed paper writer

You are a scientific paper writer. You have written a LaTeX paper below. Your task is to fix any LaTeX errors or quality issues identified in the reflection. Do NOT hallucinate results, hardware specs, or citations not present in the experiment data. Do NOT replace content with stubs. Return the ENTIRE corrected LaTeX document.

B.23 Governed paper reviewer

You are an expert academic reviewer for {venue_upper}.
Review the provided LaTeX section and return ONLY valid JSON with:

- overall: str (1-2 sentences overall assessment)
- strengths: list[str] (up to 3 key strengths)
- weaknesses: list[str] (up to 3 key weaknesses)
- suggestions: list[str] (up to 3 concrete improvement suggestions)
- accept_recommendation: str (one of: strong_accept, accept, weak_accept, reject)

Be concise and technical. No markdown fences.
Reproducibility criterion: flag as weakness any experimental detail that cannot be independently reproduced from the description alone - e.g. environment-specific identifiers (cluster names, node IDs, organization names, file paths). Hardware must be described by architecture and specifications only, not by the name of the system or organization that owns it.

References

- [1] Alex Iacob et al. *The Red Queen Gödel Machine: Co-Evolving Agents and Their Evaluators*. 2026. arXiv: 2606.26294 [cs.LG]. URL: <https://arxiv.org/abs/2606.26294>.
- [2] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. DOI: 10.48550/arXiv.2303.17651. arXiv: 2303.17651 [cs.CL]. URL: <https://arxiv.org/abs/2303.17651>.
- [3] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI]. URL: <https://arxiv.org/abs/2303.11366>.
- [4] Yuntao Bai et al. *Constitutional AI: Harmlessness from AI Feedback*. 2022. arXiv: 2212.08073. URL: <https://arxiv.org/abs/2212.08073>.
- [5] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. *AI Control: Improving Safety Despite Intentional Subversion*. 2023. arXiv: 2312.06942. URL: <https://arxiv.org/abs/2312.06942>.
- [6] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: <https://arxiv.org/abs/2408.06292>.
- [7] Chris Lu et al. “Towards End-to-End Automation of AI Research”. In: *Nature* 651.8107 (2026), pp. 914–919. DOI: 10.1038/s41586-026-10265-5. URL: <https://doi.org/10.1038/s41586-026-10265-5>.
- [8] Zhengyao Jiang et al. *AIDE: AI-Driven Exploration in the Space of Code*. 2025. arXiv: 2502.13138 [cs.AI]. URL: <https://arxiv.org/abs/2502.13138>.
- [9] Samuel Schmidgall et al. *Agent Laboratory: Using LLM Agents as Research Assistants*. 2025. DOI: 10.18653/v1/2025.findings-emnlp.320. arXiv: 2501.04227 [cs.AI]. URL: <https://arxiv.org/abs/2501.04227>.
- [10] Jun Shern Chan et al. *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*. 2024. DOI: 10.48550/arXiv.2410.07095. arXiv: 2410.07095 [cs.LG]. URL: <https://arxiv.org/abs/2410.07095>.
- [11] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.
- [12] Yujia Li et al. *Competition-Level Code Generation with AlphaCode*. 2022. DOI: 10.1126/science.abq1158. arXiv: 2203.07814 [cs.PL]. URL: <https://arxiv.org/abs/2203.07814>.
- [13] Zhuoyang Qian et al. *Story2Proposal: A Scaffold for Structured Scientific Paper Writing*. 2026. arXiv: 2603.27065 [cs.CL]. URL: <https://arxiv.org/abs/2603.27065>.
- [14] Giulio Starace et al. *PaperBench: Evaluating AI’s Ability to Replicate AI Research*. 2025. DOI: 10.48550/arXiv.2504.01848. arXiv: 2504.01848 [cs.AI]. URL: <https://arxiv.org/abs/2504.01848>.
- [15] Jerome H. Saltzer and Michael D. Schroeder. “The Protection of Information in Computer Systems”. In: *Proceedings of the IEEE* 63.9 (1975), pp. 1278–1308. DOI: 10.1109/PROC.1975.9939. URL: <https://doi.org/10.1109/PROC.1975.9939>.
- [16] Fred B. Schneider. “Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial”. In: *ACM Computing Surveys* 22.4 (1990), pp. 299–319. DOI: 10.1145/98163.98167. URL: <https://doi.org/10.1145/98163.98167>.
- [17] Yohei Nakajima. *The Log is the Agent: Event-Sourced Reactive Graphs for Auditable, Forkable Agentic Systems*. 2026. arXiv: 2605.21997. URL: <https://arxiv.org/abs/2605.21997>.
- [18] Fred B. Schneider. “Enforceable Security Policies”. In: *ACM Transactions on Information and System Security* 3.1 (2000), pp. 30–50. DOI: 10.1145/353323.353382. URL: <https://doi.org/10.1145/353323.353382>.